

DoodleCrew Project Spec

1. Project Summary

DoodleCrew is a simple real-time collaborative drawing web app.

The core experience should feel extremely easy and instant:

- Open the app.
- Start drawing immediately.
- Create a room with one click.
- Share a link.
- Friends join instantly.
- Everyone can draw together, see each other's cursors, and use the board for anything: explaining ideas, quick sketches, games like tic-tac-toe or hangman, casual doodling, planning, or collaborative image markup.

The app should prioritize simplicity, speed, and low friction over becoming a full productivity suite like Excalidraw or Miro.

2. Product Philosophy

The main differentiator is extreme simplicity.

This app should not feel like a complex whiteboard tool. It should feel more like opening a shared doodle lobby.

Guiding principles:

- No account required for MVP.
- No onboarding wall.
- No complicated dashboard.
- Drawing should work immediately.
- Creating and joining rooms should be nearly instant.
- Collaboration should feel playful and alive.
- Avoid scope creep into a giant all-in-one whiteboard app.

The app can eventually support more serious use cases, but the MVP should stay focused on being the fastest and easiest way to draw with someone online.

3. Target Use Cases

Primary Use Cases

- Quick collaborative sketching with friends.

- Explaining an idea visually during a call.
- Playing simple drawing games.
- Annotating or drawing over an uploaded/pasted image.
- Casual shared doodling.
- Sharing a temporary room link for instant collaboration.

Future Use Cases

These are not MVP requirements, but they are natural extensions:

- Collaborative coloring book mode.
- Dota 2 map drawing mode.
- Shared classroom/teaching board.
- Saved boards.
- Templates.
- Custom room backgrounds.
- Image annotation workflows.

4. MVP Scope

Must Have

- Single-page web app.
- Blank canvas users can draw on immediately.
- Basic drawing tool.
- Eraser tool.
- Color picker.
- Brush size picker.
- Clear board button.
- Create room button.
- Join room via shareable URL.
- Real-time drawing sync between users in the same room.
- Real-time cursor positions for users in the same room.
- Lightweight user identity per room, such as random display name and color.
- Room state should sync when a new user joins.
- Basic image support: paste or upload an image onto the canvas.
- Responsive layout for desktop first, with reasonable mobile/tablet support if practical.

Should Have

- Undo/redo for local user drawing actions.
- Basic object/layer model or stroke history so the board can be reconstructed.
- Room member list or simple active user indicator.
- Copy invite link button.
- Simple landing/start UI.
- Error handling for disconnected users.
- Automatic reconnect behavior.

Nice to Have

- Text tool.
- Shapes: line, rectangle, circle, arrow.
- Pan and zoom.
- Export board as PNG.
- Save board locally.
- Temporary room expiration.
- Donation button.

Not MVP

- User accounts.
- Paid plans.
- Teams/workspaces.
- Complex permissions.
- Infinite canvas with advanced scene management.
- Full Excalidraw-style object editing.
- Comments.
- Version history.
- Enterprise features.

5. Recommended Tech Stack

Frontend

Preferred option:

- Vue 3
- Vite
- TypeScript
- Pinia for state management if needed
- Tailwind CSS for styling
- vue-konva or Konva.js for canvas drawing

Notes:

- Use Vue because the builder is already more familiar with Vue and this project benefits from fast execution.
- React could be a valid learning choice, but for this app the priority is shipping a smooth real-time experience. Vue is the better practical choice unless the explicit goal changes to learning React.
- If packages already exist in the starter project, check before reinstalling.

Recommended frontend packages:

- `socket.io-client`
- `konva`
- `vue-konva` if using Vue bindings

- `pinia` if app state becomes complex
- `nanoid` for client IDs or temporary local IDs
- `@vueuse/core` for useful composition utilities

Backend

Recommended option:

- Node.js
- TypeScript
- Express or Fastify
- Socket.IO

Notes:

- Socket.IO uses WebSockets when possible, but also includes fallbacks, reconnection behavior, rooms, events, and quality-of-life features.
- For a project like this, Socket.IO is easier and safer than raw WebSocket implementation.
- Fastify is a good option if performance is a priority, but Express is totally fine for MVP.

Recommended backend packages:

- `socket.io`
- `express` or `fastify`
- `nanoid`
- `zod` for validating event payloads
- `dotenv`
- `cors`

Database

Recommended MVP options:

Option A: No database at first

Use in-memory room state on the server.

Best for:

- Fastest MVP.
- Temporary rooms.
- No account system.
- No persistent saved boards.

Downside:

- Rooms vanish on server restart.
- Scaling across multiple servers requires shared state later.

Option B: MySQL or PostgreSQL

Use relational database for persistent rooms, saved boards, and future monetization/account features.

Best for:

- Familiarity.
- Saved boards.
- Future accounts.
- More structured data.

Recommendation:

Start with in-memory state for MVP. Add MySQL or Postgres when persistence becomes necessary.

Since the builder already knows MySQL, MySQL is fine. PostgreSQL is also a great option but not required.

File Storage

For image uploads:

- MVP can store images as temporary in-memory/blob URLs or local dev uploads.
- Production should use S3-compatible object storage.

Recommended providers:

- AWS S3
- Cloudflare R2
- DigitalOcean Spaces

Recommendation:

Use Cloudflare R2 or DigitalOcean Spaces if avoiding AWS complexity. S3-style APIs are widely supported.

6. Real-Time Architecture

Use Socket.IO rooms.

Each whiteboard room maps to a Socket.IO room.

Example room URL:

```
/room/:roomId
```

Example socket flow:

1. User opens app.

2. Client creates a local anonymous user identity.
3. User clicks "Create room."
4. Server generates room ID.
5. Client navigates to `/room/:roomId`.
6. User shares link.
7. Another user opens link.
8. Client connects to socket server and emits `room:join`.
9. Server adds socket to room.
10. Server sends current room state to new user.
11. Client renders current board.
12. Users broadcast drawing/cursor events to others in room.

7. Core Data Model

User Presence

```
type RoomUser = {  
  id: string;  
  socketId: string;  
  name: string;  
  color: string;  
  cursor?: {  
    x: number;  
    y: number;  
  };  
};
```

Stroke

```
type Stroke = {  
  id: string;  
  userId: string;  
  points: number[]; // [x1, y1, x2, y2, ...]  
  color: string;  
  size: number;  
  tool: 'pen' | 'eraser';  
  createdAt: number;  
};
```

Image Object

```
type ImageObject = {  
  id: string;  
  userId: string;
```

```
url: string;
x: number;
y: number;
width: number;
height: number;
createdAt: number;
};
```

Room State

```
type RoomState = {
  id: string;
  users: RoomUser[];
  strokes: Stroke[];
  images: ImageObject[];
  createdAt: number;
  updatedAt: number;
};
```

8. Socket Events

Connection Events

room:create

Client asks server to create a new room.

Server returns:

```
{
  roomId: string;
}
```

room:join

Client joins a room.

Payload:

```
{
  roomId: string;
  user: {
    id: string;
```

```
    name: string;
    color: string;
  }
}
```

Server responds with current room state.

room:state

Server sends full room state to a joining user.

room:user-joined

Broadcast when a new user joins.

room:user-left

Broadcast when a user disconnects.

Cursor Events

cursor:move

Client sends cursor position.

Payload:

```
{
  roomId: string;
  userId: string;
  x: number;
  y: number;
}
```

Server broadcasts to other users in the room.

Important:

- Cursor updates should be throttled.
- Send around 20 to 30 updates per second max.
- 30 FPS is usually enough.
- 15 to 20 FPS may be enough for cursor presence.

Drawing Events

`stroke:start`

Sent when user begins a stroke.

Payload:

```
{
  roomId: string;
  stroke: Stroke;
}
```

`stroke:update`

Sent while user is drawing.

Payload:

```
{
  roomId: string;
  strokeId: string;
  points: number[];
}
```

Important:

- Do not send every raw mousemove event.
- Batch points and send updates at a fixed interval, such as every 16ms to 50ms.
- Render locally immediately for the drawing user.
- Remote users receive updates and render the stroke incrementally.

`stroke:end`

Sent when user completes a stroke.

Payload:

```
{
  roomId: string;
  strokeId: string;
  finalPoints: number[];
}
```

Server stores completed stroke in room state.

Board Events

`board:clear`

Clears the board for everyone.

Payload:

```
{
  roomId: string;
  userId: string;
}
```

Image Events

`image:add`

Adds an uploaded or pasted image to the room.

Payload:

```
{
  roomId: string;
  image: ImageObject;
}
```

For MVP, image upload handling can be simple. In production, upload image to object storage first, then broadcast the resulting image URL.

9. Performance Strategy

The app should feel smooth locally first.

Important rule:

The local user's drawing should never wait for the server.

When a user draws:

1. Render the stroke locally immediately.
2. Collect points into a small buffer.
3. Send buffered points over the socket at a fixed interval.

4. Other users render the received points.
5. On stroke end, send final stroke data.

Throttling

Throttling does not mean making drawing choppy.

Recommended approach:

- Capture pointer events normally.
- Draw locally immediately.
- Batch network updates.
- Send updates every 16ms to 50ms depending on performance.
- Cursor movement can be throttled more aggressively than drawing.

Suggested rates:

- Local drawing: immediate.
- Remote stroke updates: 20 to 60 FPS depending on load.
- Cursor updates: 15 to 30 FPS.
- Presence updates: event-based only.

Prioritization

Drawing data is more important than cursor data.

If performance becomes an issue:

1. Keep local drawing instant.
2. Prioritize active stroke updates.
3. Reduce cursor update frequency.
4. Reduce point density using point simplification.
5. Limit room size.
6. Add room-level scaling.

Point Simplification

Avoid storing/sending too many points.

Strategies:

- Ignore points too close to the previous point.
- Use distance threshold, such as only adding a new point every 2 to 5 pixels.
- Optionally use a simplification algorithm later.

Room Limits

For MVP, consider a soft room limit.

Example:

- 1 to 10 users: ideal.
- 10 to 25 users: likely okay with throttling.
- 25+ users: may need stronger limits or optimizations.

This app is meant for small group collaboration, not giant public lobbies.

10. Scaling Considerations

MVP

Single backend server with in-memory room state.

This is okay for early testing.

Production Scaling

If the app grows:

- Use Redis for Socket.IO adapter.
- Store room state in Redis or database.
- Use sticky sessions if required by hosting/load balancer.
- Split rooms across instances.
- Use object storage for images.
- Add cleanup jobs for inactive rooms.

Recommended later additions:

- Redis for room state and pub/sub.
- PostgreSQL or MySQL for persistent boards/accounts.
- Background job for deleting old rooms/images.

11. Persistence Strategy

MVP Persistence

Temporary rooms only.

- Room exists while active.
- Room can expire after inactivity.
- Room state can be stored in memory.

Example expiration:

- Delete room after 1 hour of inactivity.
- Or delete after 24 hours if keeping things generous.

Future Persistence

Potential saved board features:

- Save board by link.
- Download as PNG.
- Save to account.
- Duplicate board.
- Recent boards.

These require a database.

12. Image Upload Strategy

MVP

Support paste/upload image if feasible.

Implementation options:

1. Convert pasted image to object URL locally and broadcast base64 data.
2. Upload image to backend temporarily and serve from local storage.
3. Upload image directly to object storage and broadcast URL.

Recommended MVP:

- If fast local prototype: use base64 or temporary backend upload.
- For production: use object storage.

Avoid storing huge base64 images in socket room state long-term.

Production

Use object storage.

Flow:

1. User uploads or pastes image.
2. Client requests upload URL from backend.
3. Client uploads image to storage.
4. Backend/client broadcasts final image URL to room.
5. Everyone renders image on canvas.

13. Frontend UX Requirements

Main Screen

Should include:

- Canvas taking most of the screen.
- Minimal toolbar.
- Create room or share room button.
- Current tool/color/brush size.
- Room users/cursors.

Toolbar

MVP toolbar:

- Pen
- Eraser
- Color
- Brush size
- Upload image
- Clear board
- Copy room link

Keep UI minimal. The app should not feel like Photoshop had a baby with a spreadsheet.

Room Experience

When in a room:

- Show invite/copy link button.
- Show active users somehow.
- Show remote cursors with names/colors.
- Users should be able to start drawing immediately.

14. Backend Responsibilities

Backend should:

- Create rooms.
- Join users to rooms.
- Track room users.
- Track room state.
- Broadcast cursor updates.
- Broadcast stroke updates.
- Store completed strokes in room state.
- Handle disconnects.

- Clean up inactive rooms.
- Validate socket event payloads.

Backend should not:

- Block local drawing.
- Do heavy rendering.
- Store unnecessary high-frequency cursor history.

15. Suggested Project Structure

This can be a monorepo or two separate apps.

Recommended monorepo:

```
/doodlecrew
  /apps
    /web
      Vue app
    /server
      Node Socket.IO server
  /packages
    /shared
      Shared TypeScript types
```

Simpler structure:

```
/doodlecrew
  /client
  /server
```

For speed, the simple structure is fine.

16. Deployment Options

Simple MVP Deployment

Frontend:

- Vercel
- Netlify
- Cloudflare Pages

Backend:

- Render
- Railway
- Fly.io
- DigitalOcean App Platform

Important:

The backend must support long-lived WebSocket connections.

Recommended MVP Hosting

- Frontend on Vercel.
- Backend on Render, Railway, Fly.io, or DigitalOcean.

If using Vercel serverless functions for backend, be careful: serverless is usually not ideal for persistent WebSocket servers.

17. Monetization Notes

For now, keep monetization light.

Best fit:

- Donation button.
- Buy Me a Coffee / Ko-fi / GitHub Sponsors.

Avoid early paid tiers because the app's core appeal is free, instant collaboration.

Possible future paid features:

- Saved boards.
- Custom room URLs.
- Longer room history.
- Larger image uploads.
- Private persistent rooms.
- Template packs.
- Team spaces.

But do not build these until the core experience is fun and smooth.

18. MVP Build Order

Phase 1: Local Drawing

- Set up Vue/Vite app.

- Add canvas drawing with Konva.
- Add pen tool.
- Add color and brush size.
- Add eraser.
- Add clear board.

Phase 2: Rooms and Sockets

- Set up Node backend.
- Add Socket.IO server.
- Add room creation.
- Add room joining by URL.
- Add anonymous user identity.
- Add room state sync.

Phase 3: Real-Time Drawing

- Broadcast stroke start/update/end.
- Render remote strokes.
- Store completed strokes server-side.
- Sync full board state to new users.

Phase 4: Cursors and Presence

- Broadcast throttled cursor positions.
- Render remote cursors.
- Add user joined/user left handling.

Phase 5: Images

- Add image upload or paste.
- Render image on canvas.
- Broadcast image object to room.
- Decide temporary upload strategy.

Phase 6: Polish

- Copy invite link.
- Reconnect handling.
- Basic responsive layout.
- Loading/error states.
- Export PNG if easy.
- Donation button.

19. Important Implementation Notes for Coding Agent

- Check existing project setup before installing packages.
- Use Vue if the project is already Vue-based.
- Use TypeScript unless there is a strong reason not to.

- Prefer simple, readable code over over-engineered abstractions.
- Keep MVP focused.
- Do not build accounts, payments, workspaces, or dashboard features unless explicitly requested.
- Real-time drawing should use optimistic local rendering.
- Network updates should be batched/throttled.
- Cursor updates should be throttled separately from drawing updates.
- Validate socket payloads on the server.
- Avoid broadcasting massive payloads on every pointer event.
- Store strokes as data, not as canvas pixels.
- New users should receive existing room state and render it.
- Room cleanup should exist even if basic.
- Image uploads can be minimal for MVP but should be designed so object storage can replace it later.

20. Open Decisions

These should be decided before or during implementation:

1. Exact app name: DoodleCrew is the current favorite.
2. Vue vs React: recommendation is Vue for speed.
3. Database: recommendation is no database for MVP, add MySQL/Postgres later.
4. Image storage: temporary MVP approach first, object storage later.
5. Room expiration: choose 1 hour, 24 hours, or another inactivity window.
6. Mobile support level: desktop-first vs full mobile drawing support.
7. Whether to support pan/zoom in MVP.
8. Whether room state should survive server restart in MVP.

21. Success Criteria

The MVP is successful if:

- A user can open the app and draw instantly.
- A user can create a room and copy a link.
- Another user can join and draw within seconds.
- Both users see each other's drawing in real time.
- Both users see each other's cursors.
- Drawing feels smooth locally.
- Remote drawing feels reasonably smooth.
- The UI is simple enough that no explanation is needed.

22. What Not To Build Yet

Do not accidentally build a giant whiteboard SaaS.

Avoid for MVP:

- Accounts.
- Billing.

- Permissions.
- Teams.
- Admin dashboard.
- Complex document organization.
- Full object editing system.
- Advanced shape libraries.
- Comment threads.
- AI features.
- Video/audio chat.

The magic is speed and simplicity.

Build the fun shared drawing room first.